IO模型：输入输出模型

我们知道linux中输入输出不是直接从键盘获取数据，也不是直接往终端输出(缓冲区)

那么我们可以把IO理解为两个步骤：

1. 等待IO事件的就绪。比如：读，我们要等待缓冲区有数据可读 写，需要等待缓冲区有空间可写
2. 真正意义上的数据的迁移(涉及到用户态和内核态的切换)

# 1.linux中五种IO模型

1. 阻塞模型

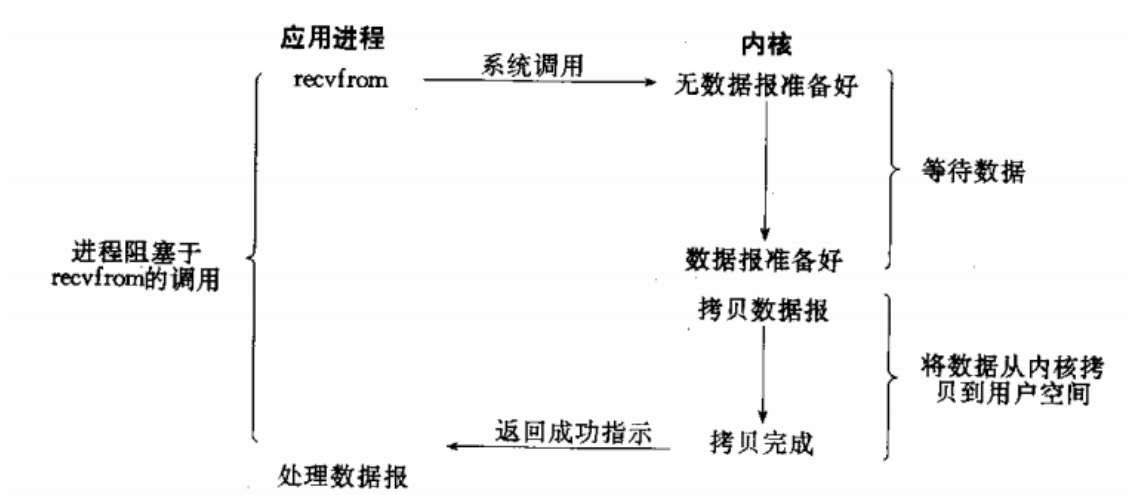



图 6.1 阻塞 I/O 模型

   缺点：如果是在单线程环境下，由于是阻塞地获取结果，只能有一个客户端连接。如果是多线程环境下，需要不断地新建线程来接收客户端。这样会浪费大量的空间和资源。

2. 非阻塞模型：意味着程序无需等待结果，持续运行
   - 读：如果有数据，则读取数据

     如果没有数据，就立即返回
   - 写：如果有空间，则直接写入

     如果没有空间，即立即返回

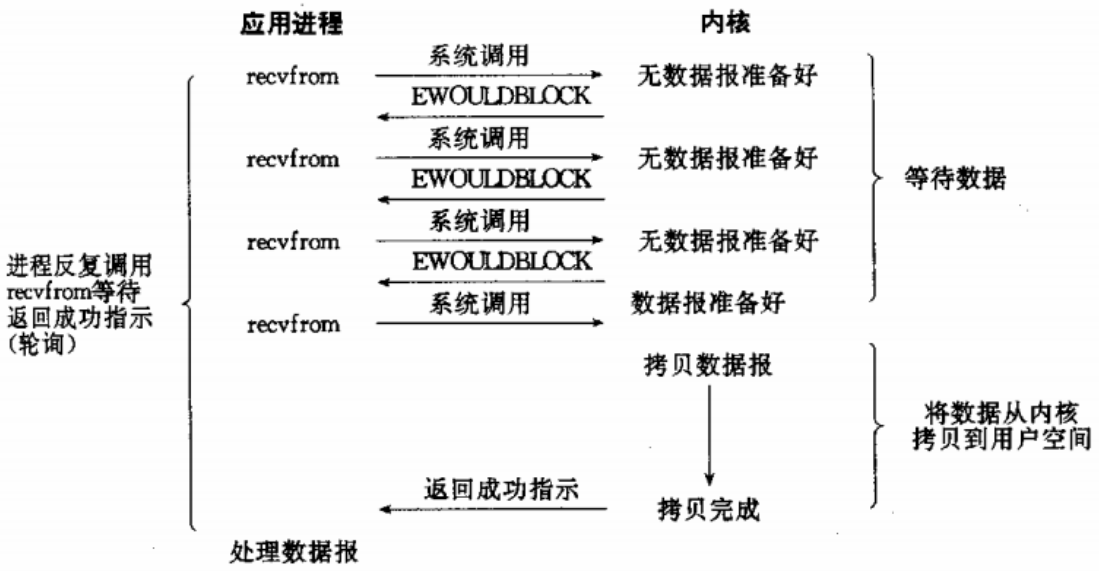



图 6.2 非阻塞 I/O 模型

缺点：采用轮询，占用cpu, 不及时

虽然这种方式不会阻塞，但是有一个缺点，这种方式相对于阻塞IO来说，效率更低，会浪费大量的cpu的时间

3. IO多路复用
   - 允许同时对多个IO事件进行控制，同时在监控多个"文件描述符"

     同时监听多个文件描述符是否就绪(是否可写/可读/出错)
   - IO多路复用实现的机制是通过select/poll/epoll函数来实现

     先有select,再有了poll/epoll,由两个不同的组织写的，但是作用是类似

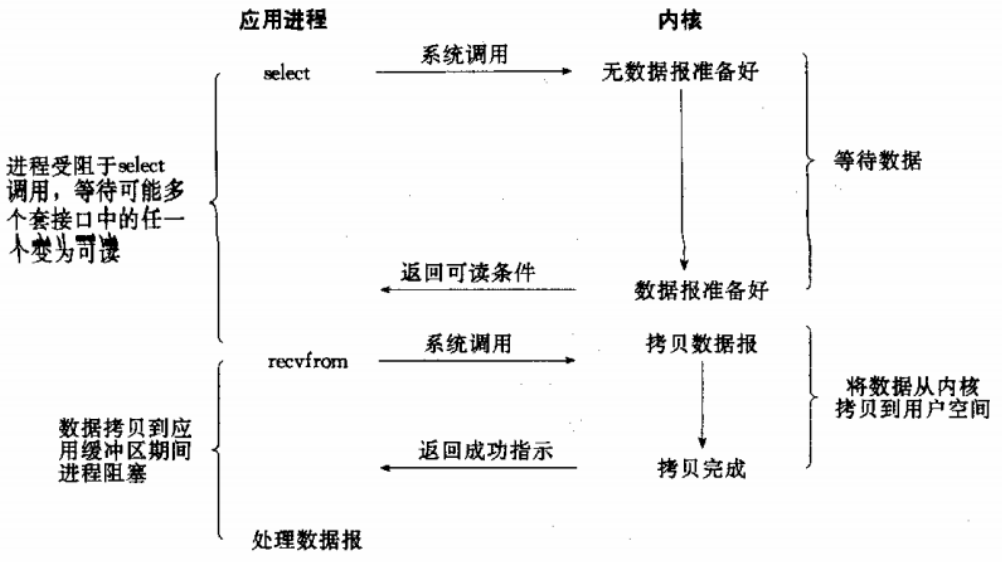



图 6.3 I/O 复用模型

4. 信号驱动IO

   如果有IO事件就绪，就可以发送一个信号给应用程序，进行数据的处理

就相当于注册一个信号处理函数，当数据准备好了，就给我发送一个信号，我捕捉这个信号之后，就可以直接去信号处理函数里面去获取数据即可

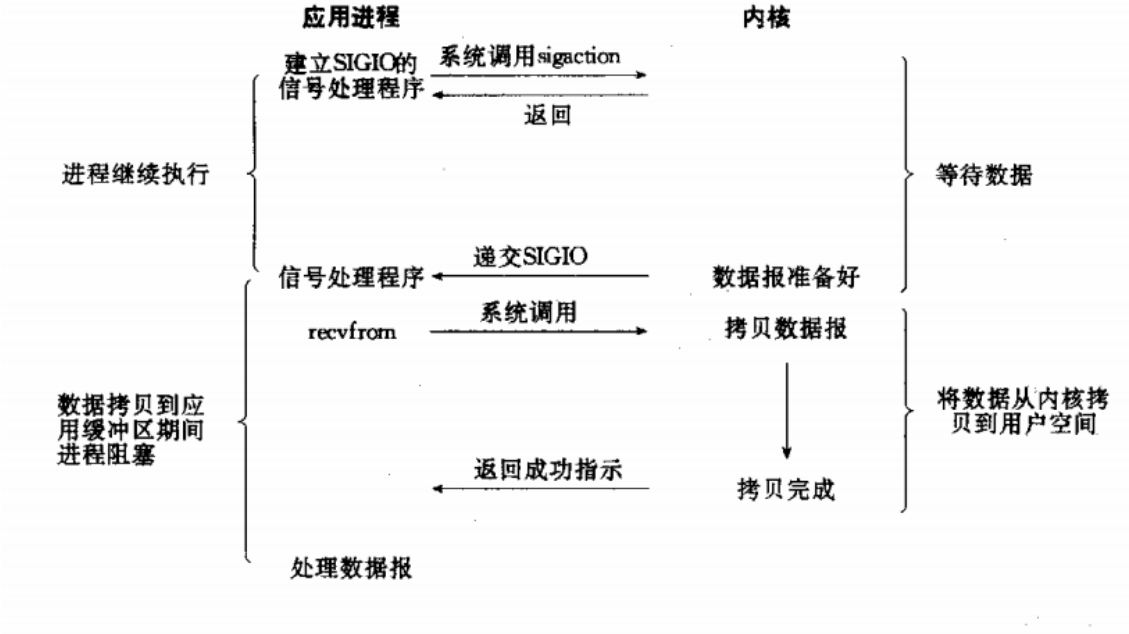



图 6.4　信号驱动 I/O 模型

5. 异步IO

相当于E去钓鱼，但是E还有其他的事情要做，所有它打电话叫来了它的好朋友F帮他看钩子，一旦鱼上钩了，F就会起钩， 然后打电话告诉E鱼钓上来了

进程E仅仅就是对数据的请求，而数据的监控和数据的迁移都是由F完成的

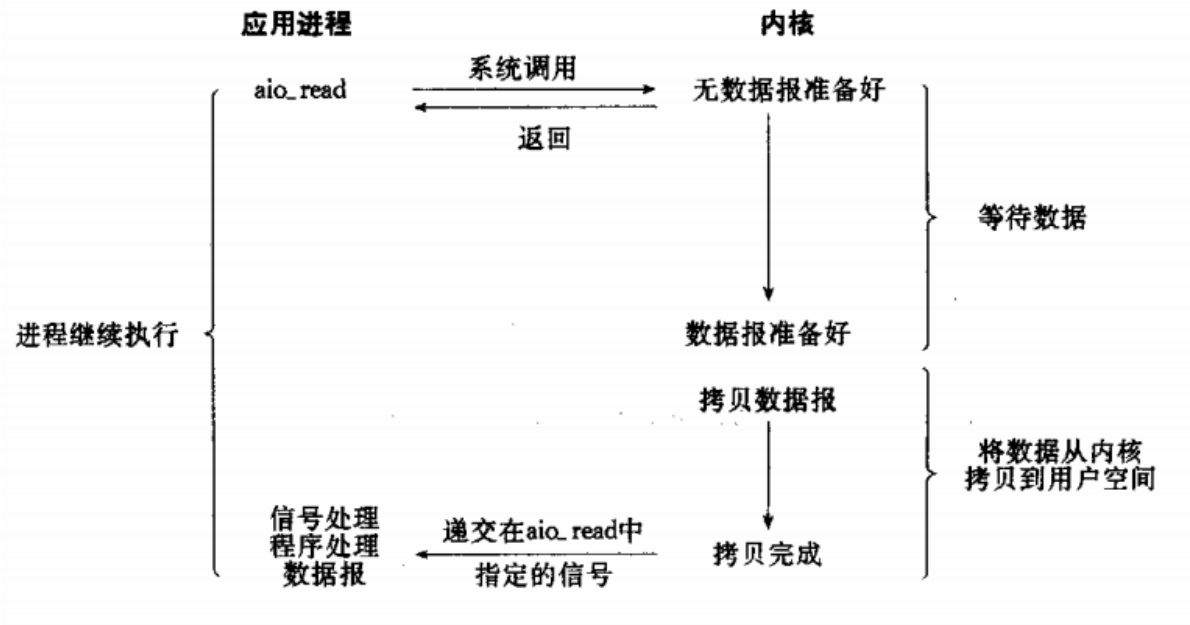


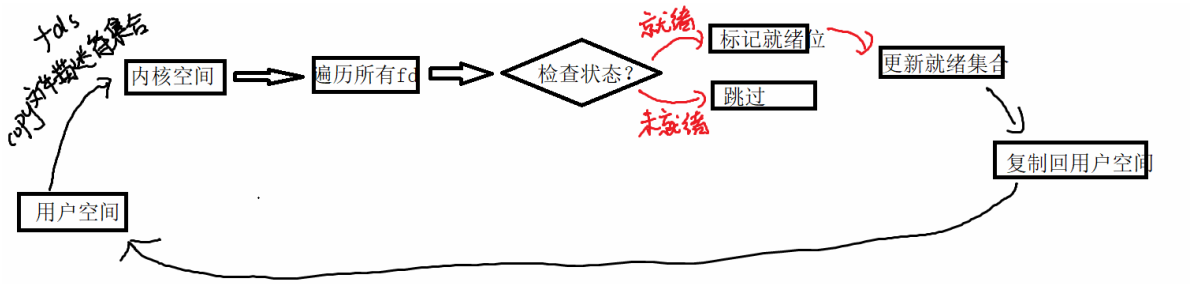

图 6.5　异步 I/O 模型

# 2.IO多路复用的实现方式

什么是多路复用?

就是预先把要监听的文件描述符加入到集合，然后在规定的时间内或无限时间内进行等待。如果在规定的时间内，集合中文件描述符没有数据的变化，则说明超时接收，并且进行下一次规定时间再等待。一旦集合中文件描述符有数据变化，则其他的没有数据变化的文件描述符就会被剔除到集合外，并且再次进入下一次等待状态

select原理如图所示:




特点：可以实现多个文件描述符进行监听。一般在网络服务器中，可以并发的处理多个客户端的连接和请求

并发：创建进程/线程/多路复用

1. select

```
#include <sys/select.h>

typedef /* ... */ fd_set;

int select(int nfds, fd_set *readfds,
fd_set *writefds,
fd_set *exceptfds,
struct timeval *timeout);
@nfds:三个集中中最大的文件描述符+1,因为这个参数会告诉内核检测前多少个文件描述符
@readfds：可读文件描述符集合
@writefds：可写文件描述符集合
@exceptfds：错误文件描述符集合
@timeout：设置阻塞等待的时间，有3种情况
        1.设置NULL，永远等待，这个函数阻塞的(无限等待，直到有文件描述符发送变化)
        2.设置timeout,等待一个固定的时间
            struct timeval
            {
                long tv_sec;//秒
                long tv_usec;//微妙
            };
            eg：等待5s
                struct timeval tv;
                tv.tv_sec = 5;
                tv.tv_usec = 0;
            在这段时间中，若select正常返回，timeout的值，被设置成剩余的秒数
            若想每次调用select都等待固定的时间，那么需要再调用前重新设置timeout
        3.设置0 非阻塞
返回值：
    >0 表示已经就绪的文件描述符的个数
        FD_ISSET去判断到底是哪个文件描述符就绪
    =0  超时
    <0  出错

void FD_CLR(int fd, fd_set *set);//把fd指定的文件从set指定的集合中移除
int  FD_ISSET(int fd, fd_set *set);//判断fd是否就绪
void FD_SET(int fd, fd_set *set);//把fd加入到指定的集合中
void FD_ZERO(fd_set *set);//把set指定的集合清空
```

eg：tcp服务器代码步骤

```
//创建套接字   sockfd
//绑定
```

```
//监听
int clientfds[50] = {0};//保存所有的客户端的fd
int client_count = 0;//客户端的数量
int maxfd = sockfd;//初始化最大的文件描述符
while(1)
{
    //1.定义一个集合
    fd_set readfds;//可读文件描述符集合
    //2.清空集合
    FD_ZERO(&readfds);
    //3.把你需要要监听的fd加入到可读集合
    FD_SET(sockfd, &readfds);//添加服务器的fd
    for(int i = 0;i<client_count;i++)
    {
        FD_SET(clientfds[i], &readfds);//添加全部客户端的fd
    }
    //4.设置超时时间 5s
    struct timeval tv;
    tv.tv_sec = 5;
    tv.tv_usec = 0;
    //5.select
    int ret = select(maxfd+1,&readfds,NULL,NULL,&tv);
    //6.处理结果
    if(ret > 0)//检查具体的fd
    {
        if(FD_ISSET(sockfd,&readfds))//sockfd是socket的返回值 sockfd就绪
        {
            //代表有客户端想你发起连接
            int newclientfd = accept(.....);
            clientfds[client_count] = newclientfd;
            client_count++;//客户端的数量+1
            if(maxfd < newclientfd)
            {
                //更新maxfd
            }
        }
        for(int i = 0;i<client_count;i++)
        {
            if(FD_ISSET(clientfds[i],&readfds))//判断clientfds[i]是否就绪
            {
                //有客户端给你发送数据 read/recv/recvfrom
                //read/recv/recvfrom 返回值0 代表有客户端退出
            }
        }
    }
    else if(ret == 0)
    {
        printf("超时\n");
    }

}
```

练习：利用select实现tcp一对多的通信(只改服务器的代码)